INSTITUTO FEDERAL
São Paulo
Campus
São Paulo

SPOCPGR

Profº João Temochko

Breve História

- O primeiro computador a possuir recursos gráficos foi o Whirlwind I, desenvolvido pelo MIT (em 1950) com finalidades acadêmicas e militares.

- Sistema de monitoramento e controle de vôos que o utilizava foi desenvolvido, para defesa aérea dos USA, em 1955, que convertia as informações capturadas por radar em imagem em tubo de raios catódicos.

Breve História

- Em 1959, o termo Computação Gráfica surgiu em um projeto para simulação de vôos.

- Em 1962, a tese de Sutherland (Sketchpad- A Man-Machine Graphical Communication System), introduziu as estruturas de dados e os conceitos de computação gráfica interativa que levaram ao desenvolvimento dos primeiros sistemas de projetos auxiliados por computador (Computer Aided Design-CAD).

- Na década de 1990, o OpenGL começou a ser embutido em linguagens de programação, como o C.

O Que é Computação Gráfica?

- Computação gráfica (CG) é o ramo da ciência da computação focado na criação, manipulação, armazenamento e exibição de imagens, modelos 3D e animações digitais.

- Atua principalmente em uso de vetores gráficos (normalmente usados pela Placa de Vídeo) e elementos em que precise de processamento da parte gráfica.

Vetor Gráfico

- Vetores gráficos podem ser descritos como caminhos definidos por um começo e um fim, com outros pontos, curvas e ângulos inclusos.

- Esses vetores normalmente vão resultar em uma forma ou um conjunto de formas, normalmente chamados Polígonos Gráficos.

Vetor Gráfico

- Normalmente quando falamos de Vetor Gráfico, falamos muito de linhas que se conectam, quando temos muitos vetores, a placa de vídeo (GPU) irá trabalhar cada vez mais.

- Chamamos um grupo de vetores (ou polígonos, que são comumente um conjunto de vetores que têm alguma forma) de Low Poly (Polígonos Baixos) e High Poly (Polígonos Altos).

Low Poly vs High Poly

- Low poly usa poucos polígonos para jogos rápidos e desempenho otimizado.

- High poly utiliza milhares ou milhões de polígonos para máxima riqueza de detalhes, suavidade e realismo, ideal para animações e renderizações cinematográficas.

Low Poly vs High Poly

- O Low Poly apresenta mais desempenho, pela GPU principalmente processar menos vetores.

- O High Poly é muito usado para objetos detalhados, porém são mais pesados para a GPU.

Importância

E onde queremos chegar com isso?

Importância

- Para começarmos a entender a computação gráfica, precisamos entender exatamente essa parte.

- Quando mexemos com projetos densos graficamente, precisamos entender qual nosso objetivo.

- Se precisamos de desempenho e não nos importamos com algo muito detalhista, Low Poly. Se precisamos de detalhes e não nos importamos em aumentar a GPU como requisito, High Poly.

Impacto

- O principal impacto da Computação Gráfica é na Placa de Vídeo (GPU).

- A GPU é simplesmente milhares de pequenos núcleos que possuem um propósito, mexer com shaders (sombras) e vetores.

- Uma GPU moderna também possui outras funções, como tensor cores.

Impacto

- Não é somente a GPU que é impactada por isso, a CPU e a Memória também tem um impacto na computação gráfica.

- Lembre-se: Altas cargas acontecem na GPU em parte de objetos gráficos, porém cargas como partículas e comandos ainda são prioritariamente feitos pela CPU.

Ajuda

- A computação gráfica é amplamente usada em animações (2D e 3D), Jogos e Softwares CAD.

- Não somente nestes casos, mas são a maioria.

Problemas da Computação Gráfica

- Os principais problemas da computação gráfica envolvem renderização em tempo real, flickering (objetos piscando), oclusão (gestão de objetos ocultos) e a simulação realista de iluminação e texturas.

- Esses desafios técnicos afetam a qualidade visual, o desempenho em jogos e a precisão em simulações 3D, exigindo hardware potente e algoritmos complexos.

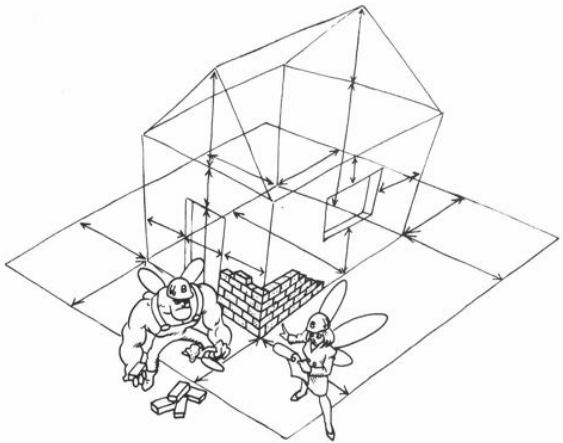
Problemas da Computação Gráfica

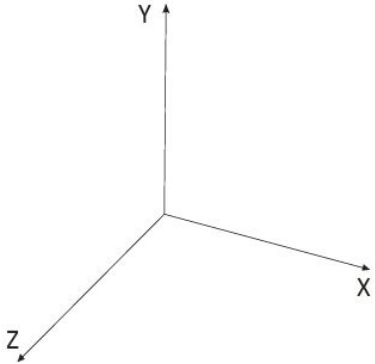
- Os principais problemas da computação gráfica envolvem renderização em tempo real, flickering (objetos piscando), oclusão (gestão de objetos ocultos) e a simulação realista de iluminação e texturas.

- Esses desafios técnicos afetam a qualidade visual, o desempenho em jogos e a precisão em simulações 3D, exigindo hardware potente e algoritmos complexos.

Modelagem Geométrica

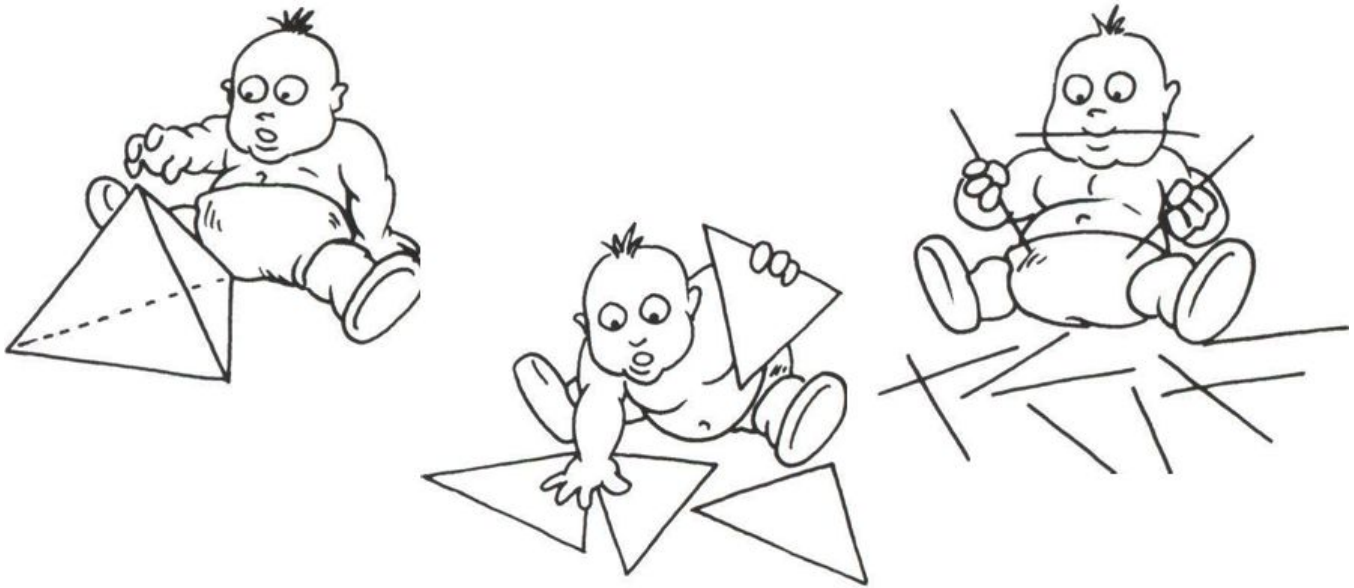
- Modelagem Geométrica estrutura e descreve dados geométricos no computador.





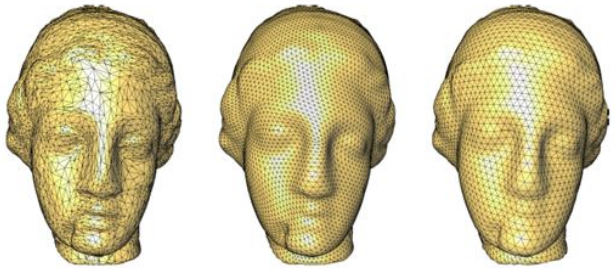
Modelagem: Representação de objetos

- São objetos são definidos por pontos, linhas e planos.

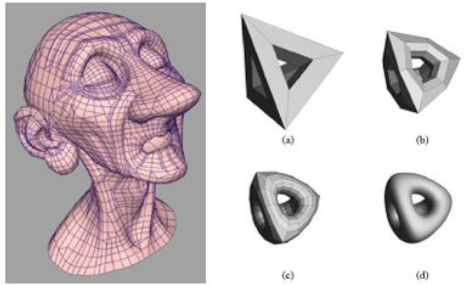


Modelagem: Representação de objetos

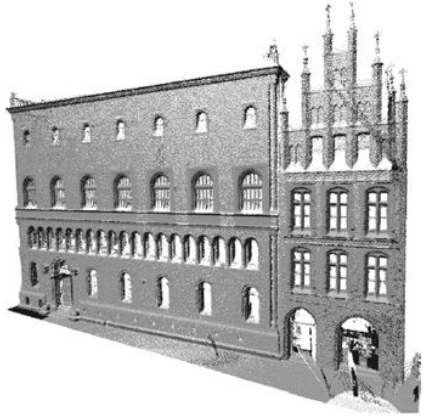
- São objetos são definidos por pontos, linhas e planos.



Triângulos



Superfícies de Subdivisão



Pontos

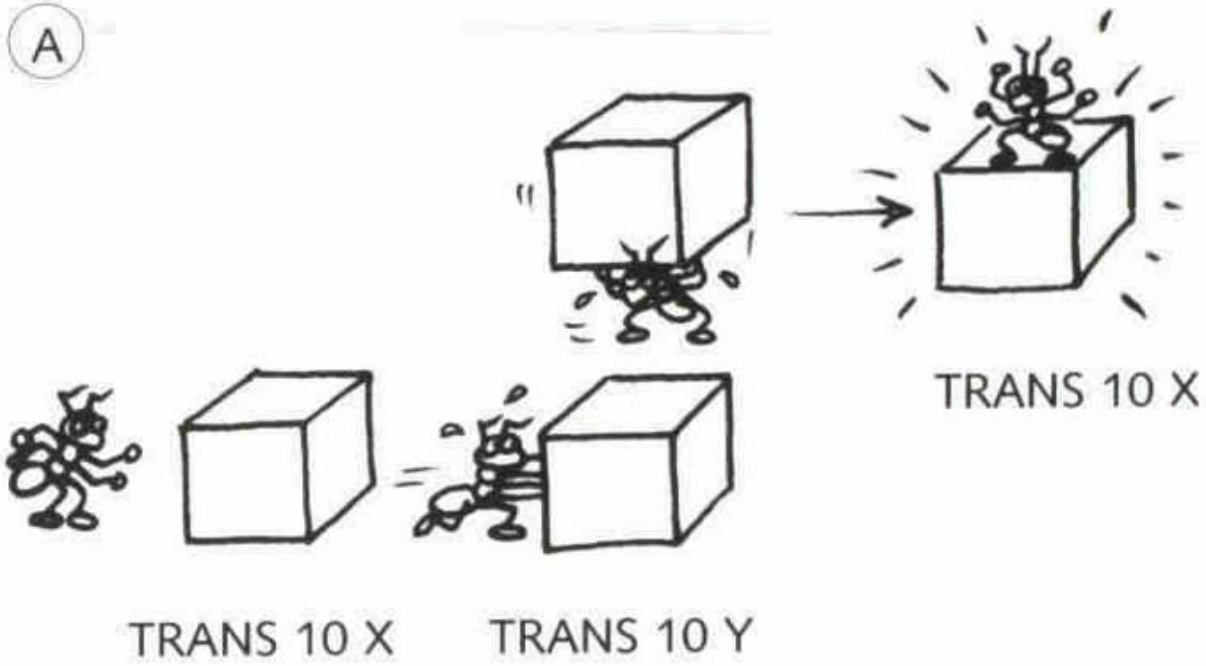
Modelagem: Transformações Básicas

- As transformações básicas seguem os eixos X,Y para 2D e X,Y,Z para 3D.

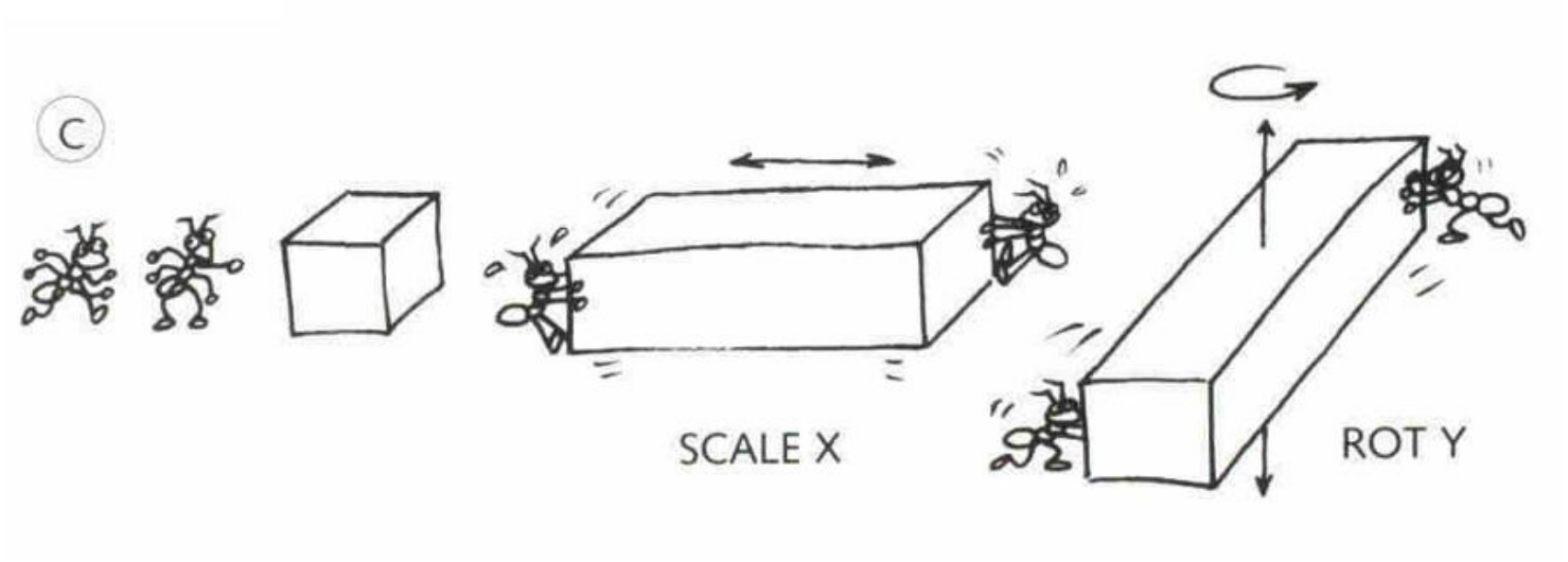
- Os eixos são um método de representar a posição de um ponto no espaço por valores numéricos.

- Isso ajuda a direcionar o objeto para onde você quer realmente que ele esteja, seja da percepção do usuário ou do cenário.

Modelagem: Transformações Básicas - Translação



Modelagem: Transformações Básicas - Escala



Jogos

- Um jogo é nada mais que um software que é renderizado em tempo real a cada segundo (chamado de Frame).

- Ele roda em um loop infinito que, enquanto o jogo estiver rodando, ele vai processar as informações e renderizar em tempo real.

- Um ponto principal é o uso em APIs Vulkan, DirectX e OpenGL.

Jogos

- A utilização dos recursos da computação gráfica em jogos englobam:
  - Partículas;
  - Objetos Renderizados ou Pré-Renderizados;
  - Colisões (muitas vezes a CPU que processa);
  - Transformação (quanto maior o objeto, mais poder de processamento ele precisará);
  - etc.

OpenGL

- O OpenGL (Open Graphics Library) é uma API gráfica multiplataforma e multi linguagem padrão da indústria, usada para renderizar gráficos vetoriais 2D e 3D acelerados por hardware via GPU.

- Ela funciona como uma ponte entre o software e os drivers da placa de vídeo, sendo amplamente adotada em jogos, CAD e realidade virtual.

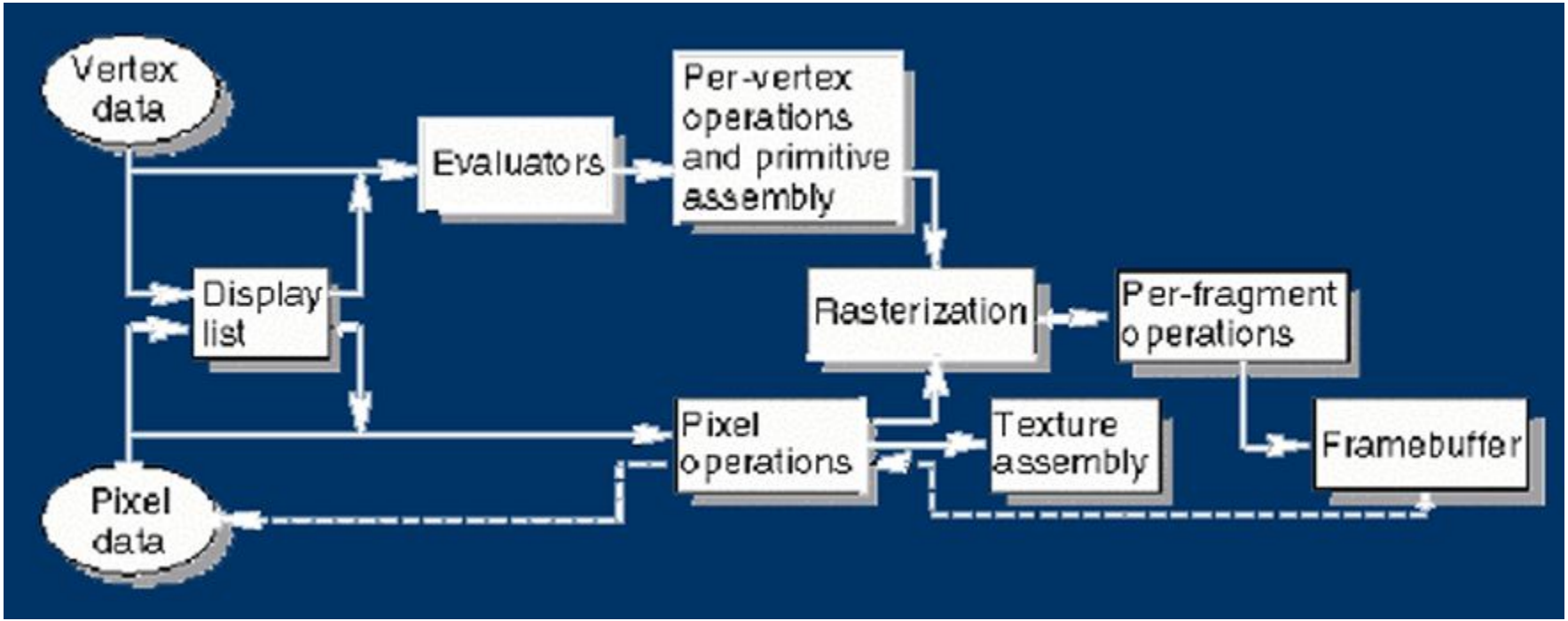
O Pipeline do OpenGL

- A maior parte das implementações do OpenGL tem uma ordem de operações a serem executadas.

- Uma serie de estágios de processos chamam o “pipeline” de renderização do OpenGL.

- O diagrama seguinte mostra como o OpenGL, obtém e processa os dados.

O Pipeline do OpenGL



Funções gráficas do OPENGL

- **Buffer de acumulação:** Trata-se de um buffer no qual múltiplos frames renderizados, podem ser compostos para produzir uma única imagem. Usado para efeitos tais como a profundidade de campo, “blur” de movimento, e de anti-aliasing da cena.

- **Alfa Blending:** Provê mecanismos para criar objetos transparentes. Usando a informação alfa, um objeto pode ser definido como algo totalmente transparente até algo totalmente opaco.

Funções gráficas do OPENGL

- **Anti-aliasing**: Um método de renderização utilizado para suavizar linhas e curvas. Esta técnica calcula a média da cor dos pixels junto à linha. Tem o efeito visual de suavizar a transição dos pixels na linha e daqueles junto à linha, assim fornecendo uma aparência mais suave.

- **FeedBack**: Um modo onde OpenGL retornará a informação geométrica processada (cores, posições do pixel, e assim por diante) à aplicação.

Sintaxe de Comandos do OpenGL

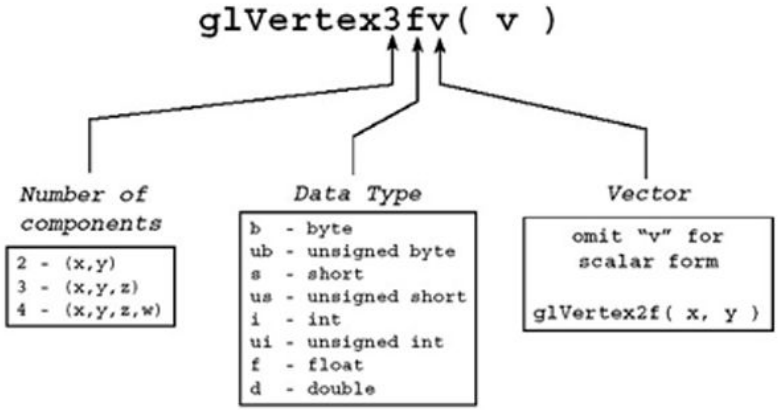
- Os comandos da API OpenGL obedecem a um padrão bem definido para especificação dos nomes de funções e constantes. Todos os comandos utilizam-se do prefixo gl em letras minúsculas.

- Similarmente, OpenGL define constantes com as iniciais GL_, em letras maiúsculas, e usa um “underscore” para separar as palavras (Ex. **GL_COLOR_BUFFER_BIT**).

- Em algumas funções algumas seqüências de caracteres extras aparecem,como sufixo, no nome destas funções (Ex. **glColor3f** e **glVertex3f**).

Sintaxe de Comandos do OpenGL

- Particularmente, o a parte “3” do sufixo indica que três argumentos são necessários; outra versão de “Color” necessita de 4 argumentos. O “f” do sufixo indica que os argumentos são do tipo números de ponto flutuante. Através destas definições para diferentes tipos de formatos e permitido ao OpenGL, aceitar dados no seu próprio formato definido.



Sintaxe de Comandos do OpenGL

- Alguns comandos OpenGL aceitam até 8 tipos diferentes de argumentos. As letras usadas como sufixo, como dito anteriormente, irão determinar o tipo de dados correspondente à padronização ISO para implementação em linguagem C.

Sintaxe de Comandos do OpenGL

Sufixo	Tipo de Dados	Tipo correspondente na linguagem C	Definição de tipo para OpenGL
b	Inteiro de 8 bits	Signed char	GLbyte
s	Inteiro de 16 bits	Short	GLshort
i	Inteiro de 32 bits	Int ou long	GLint,GLsizei
f	Ponto flutuante de 32 bits	Float	GLfloat,GLclampf
d	Ponto flutuante de 64 bits	double	GLdouble,GLclampd
ub	Inteiro não sinalizado de 8 bits	Unsigned char	GLubyte,GLboolean
us	Inteiro não sinalizado de 16 bits	Unsigned short	GLushort
ui	Inteiro não sinalizado de 32 bits	Unsigned int ou unsigned long	GLuint,GLenum,GLbitfield

Estrutura de Programas em OpenGL

- Um programa OpenGL deve conter, um mínimo de requisitos para sua perfeita execução. Normalmente alguns passos devem ser seguidos para criação destes programas. Estes passos são :
  - Declaração dos arquivos de header para o OpenGL;
  - Configurar e abrir a janela;
  - Inicializar os estados no OpenGL;
  - Registrar as funções de “callback”;
  - Renderização;
  - Redimensionamento;
  - Entradas : teclado, mouse, etc;
  - Entrar no loop de processamento de eventos.

Estrutura de Programas em OpenGL

Os arquivos de header, contém as rotinas e declarações necessárias para utilização do OpenGL/GLUT com a linguagem c/c++.

```
#include <GL/gl.h>
#include <GL/glut.h>
void main( int argc, char** argv )
{
     int mode = GLUT_DOUBLE | GLUT_RGB;
     glutInitDisplayMode( mode );
     glutInitWindowSize(400,350);
     glutInitWindowPosition(10,10);
     glutCreateWindow( argv[0] );
     init();
     glutDisplayFunc( display );
     glutReshapeFunc( resize );
     glutKeyboardFunc( key );
     glutIdleFunc( idle );
     glutMainLoop();

}
```

Estrutura de Programas em OpenGL

- As funções glutInitDisplayMode() e glutCreateWindow() compõem o passo de configuração da janela.

- O modo da janela que é argumento para a função glutInitDisplayMode(),indicam a criação de uma janela double-buffered (GLUT_DOUBLE) com o modo de cores RGBA (GLUT_RGB).

Estrutura de Programas em OpenGL

- O primeiro significa que os comandos de desenho são executados para criar uma cena fora da tela para depois rapidamente colocá-la na view (ou janela de visualização).

- Este método é geralmente utilizado para produzir efeitos de animação. O modo de cores RGBA significa que as cores são especificadas através do fornecimento de intensidades dos componentes red, green e blue separadas.

Estrutura de Programas em OpenGL

- A função glutCreateWindow() cria a janela com base no parâmetros definidos nas funções glutInitWindowSize (Tamanho da janela em pixels) e glutInitWindowPosition (coordenadas para criação da janela).

- Esta janela conterá o nome especificado em seu parâmetro de entrada.

Estrutura de Programas em OpenGL

- Em seguida é chamada a rotina init(), a qual contem a primeira inicialização do programa. Neste momento serão inicializados quaisquer estados OpenGL, que serão executados na execução do programa.

- O próximo passo será o registro das funções de callback, que estarão sendo utilizadas na execução do programa.

- Isso será feito nas próximas aulas.

INSTITUTO FEDERAL
São Paulo
Campus
São Paulo

Referências

- https://ic.ufal.br/professor/thales/OpenGL.pdf
- https://ic.ufal.br/professor/thales/icg/Aula1.pdf
- https://www.lsi.usp.br/~mkzuffo/PSI5787/vrml/modulo1/texto08.html